

7장 langchain, Agentic AI

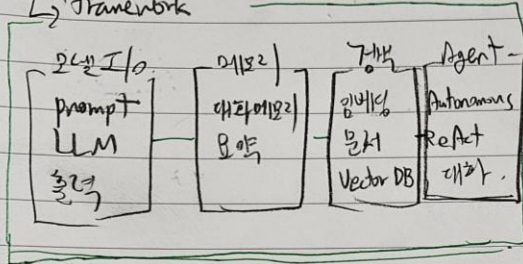
Date

Langchain

추상화를 통해 LLM 작업을 단순화해 주는 프랑기 framework 중 하나.

API를 fine tuning 안 해도 LLM에서의 출력과 경험 향상 가능.

framework



7.1 모델 I/O

양자화

LLM 모델 parameter를 표현하는데 필요한 비트 수 줄여서 원본 모델 점유한 바이트

Phi-3 대신 GGUF 모델 같은 것

원본 정보 유지, LLM의 parameter 표현에 필요한 비트 수 줄여줌

정밀도 ↓

모델이 빠르게 실행됨

작은 VRAM 필요

원본 모델만큼 저장공간 ↑

행렬적 작업은 계속 유지함. ex) 14:16

최소 4비트 사용하는 것 추천

ex) llama-cpp python

GGUF 파일로 저장

invoke 메서드

출력 생성

7.2 체인

Chain

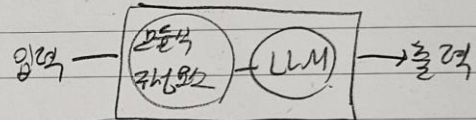
LLM을 다른 구성요소나 함께 사용하거나 다른 모델과 연결해 사용할 때 워크 플로우

LLM의 기능 확장

여러 체인 연결도 가능

7.2.1 단일체인

LLM을 구체적인 도구, prompt, 기능 등과 연결시킴



프롬프트 템플릿

LLM에 연결해 원하는 출력 얻기

LLM을 이거 사용할 때, prompt template이 없으면 사용자 prompt와 system prompt만 정의하면 됨

ex) phi-3 template

<S>: prompt 시작

<User>: 사용자 prompt의 시작

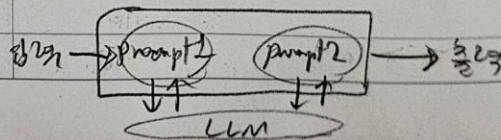
<Assistant>: 모델 출력의 시작

<end>: prompt나 모델 출력의 끝

응답에는 불필요 도록 X

샘플링 안함 → 독자들의 출력은 각각 다른 것

7.2.2 여러 템플릿 + 체인

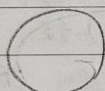


- ↳ 복잡한 prompt를 더 작은 하위 작업으로 쪼개어서 순차적으로 실행 가능
- ↳ 더 작은 prompt로 LLM을 여러번 호출하고 중간 출력 만들수 있다.
- ↳ 특정 하위작업을 처리하는 chain을 만들어라.

↳ prompt를 관리가능한 더 작은 작업으로 나누는것. 하나의 입력만 받고, 순차적으로 세 요소를 생성하는 chain 만들기.

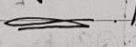
ex) 코드 예시

llm-chain = title | character | story

Template = 

story-prompt = Prompt Template

template-template, input-variables = "summary", "title", "character"

story = llmchain ()

↳ 사용자는 요약에 해당하는 짧은 prompt 하나만 입력하면 됨.

↳ 개별 라이브러리 활용 가능.

7.3 메모리

입력 → LLM → 출력

↑ 기억
입력 → LLM → 출력

- 1) 대화 버퍼 Conversation buffer
- 대화 요약 Conversation summary

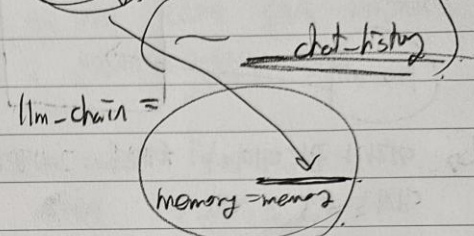
7.3.1 대화 버퍼

↳ 과거의 대화를 그대로 전달

↳ Conversation Buffer Memory 사용

↳ 과거 대화를 포함해 다 저장

ex) memory = Conversation Buffer Memory



↳ LLM이 이전의 대화만 내역을 찾을수 있게 됨.

7.3.2 윈도우 대화 버퍼

↳ 챗봇은 지금까지 나온 모든 대화 기억

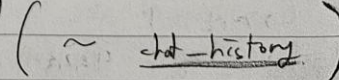
↳ 대화가 늘어나면 압력 prompt 커짐

↳ 최대토큰 개수 초과 가능

↳ 마지막 k개의 대화만 사용하며 문맥 윈도우 크기 확대하자.

↳ Conversation Buffer Window Memory 사용

ex) memory = Conversation Buffer Window Memory



↳ 대화 기록의 크기를 줄여주지만 마지막 몇개의 대화만 기억 → 긴 대화에 이상적 X

7.3.3 대화 요약

이전 문제) Conversation Buffer Memory

대화기록이 → 도록의 제한 개수에 도달

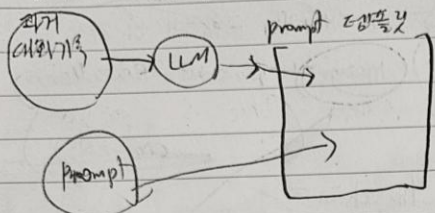
Conversation Buffer Window Memory

마지막 k개 대화만 유지

문맥 윈도우를 큰 LLM 사용 → 계산시간↑

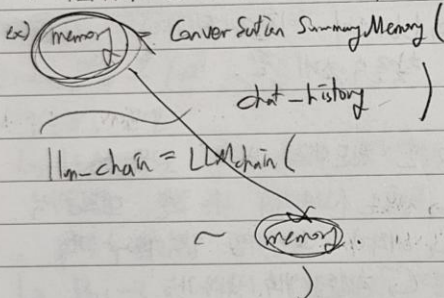
Conversation Summary Memory

현재 대화 기록 요약 → 핵심 요점 추출.



→ 대화가 끝날 때마다 체인은 그 시점까지의 대화를 요약

→ 대화 계속하면 단계마다 요약, 정보 추가



→ 가장 최근의 요약한 것 → 이전에 만든 메모리 변수 참조 가능.

→ 많은 프롬프트, 대화 기록 체계 유지에 도움.

→ 문맥 윈도우 축소해줄 수 있는 LLM 체인 방법

→ 계산시간↑

→ 구체적인 접근이 대화 기록에 저장되어야 하는 것.

1.4 Agent

• Agent: LLM이 행동을 결정할 수 있는 능력.

• Tool: Agent가 스스로 수행할 수 없는 작업을 위해 사용될 수.

• Agent type: 수행할 행동 또는 사용할 도구를 결정하는 것.

→ Agent는 목표달성을 위한 코드를 스스로 실행 가능.

→ LLM이 사용자, 실세계와 상호작용 가능.

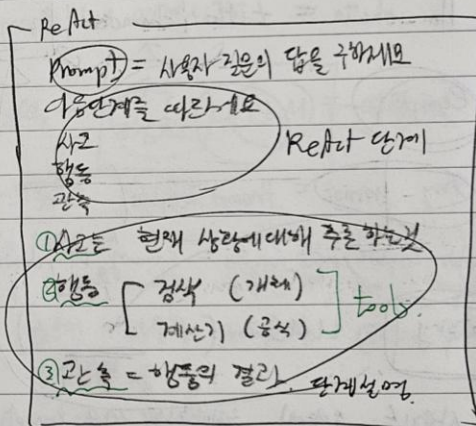
→ LLM 사용의 사용자 경험을 아래와 같이 먼저 어떻게 도구를 사용할지 결정하는 것.

1.4.1 ReAct

• 흐름) LLM이 매우 강력함.

• 행동) 논리예외 API 같은 특정도구를 사용해야 함.

• ReAct) 추론 + 행동.



1.4.2 ReAct의 ReAct.

React-template

prompt = PromptTemplate ()

tools.append (search-tool)

agent = create-react-agent (llms, tools, prompt)

• agent-executor.invoke (~)

→ but 실제로 많이 불비한지 판단해야 함.

자동화된 동작을 만드는 중간에 사람 참여 X.

→ 신뢰성을 높이려면 주의관계 시스템을

설계해야 함.

• Agent가 백플로어의 가격을 찾는

사이의 URL 변환 이 단계마다 출력물을 바꿔줌...